

Space Greenhouse × Agentic AI Hackathon 專案概念說明

1. 專案背景

本專案以「太空溫室」為核心場景，但目標並非單純將現有的智慧農業（AIoT Smart Greenhouse）搬到太空。

傳統智慧溫室主要解決：

如何根據環境與植物狀態，自動維持適合植物生長的環境？

例如：

- 溫度過高 → 啟動冷卻
- 濕度過低 → 增加灌溉
- CO₂ 濃度異常 → 啟動通風
- 光照不足 → 增加人工照明
- 透過 AI 預測植物疾病或設備異常

即使加入 AI，其核心通常仍是：

Sense → Detect/Predict → Control

然而太空環境真正困難的地方，不只是「如何把植物種好」，而是：

在無法依賴地球即時支援、資源極度有限且環境高度不確定的情況下，如何自主維持一個能支撐人類長期生存的封閉生態系統？

因此，本專案將太空溫室進一步延伸為：

Autonomous Closed-Loop Life Support & Mission Planning System

2. 核心問題

假設火星基地中有 4 名太空人，需要生存 500 天。

基地擁有有限的：

- 水
- 電力
- 種植面積
- 植物種子
- 養分
- 氧氣

- 儲備食物
- 設備

基地同時需要滿足：

- Crew 生存需求
- Calories
- Protein
- Vitamins / Minerals
- O₂ / CO₂ balance
- Water demand
- Crop growth
- Energy consumption
- Emergency reserve

系統真正需要回答的不是：

「現在溫度太高，要不要開冷氣？」

而是：

「在目前所有資源與未來風險之下，接下來應該採取什麼策略，才能讓整個 Mission 持續生存？」

3. 跨領域定位

本專案的跨領域核心不是「Space Greenhouse × Multi-Agent」。

Multi-Agent 屬於解決問題的技術架構，而不是跨領域問題本身。

真正結合的領域為：

Space Agriculture × Human Nutrition × Circular Life Support

並使用：

Agentic AI × Multi-Agent System × Digital Twin / Simulation

作為解決方案。

3.1 Space Agriculture

研究不同作物在封閉環境中的：

- 生長週期
- 光照需求

- 水需求
- 能源需求
- 溫度需求
- CO₂ 消耗
- O₂ 產生
- 可食用 biomass
- 單位面積產量

系統不只需要讓植物健康，而需要決定：

有限的種植空間應該種什麼？種多少？什麼時候種？

3.2 Human Nutrition

植物生產的最終目的並非最大化 biomass，而是支撐 Crew 生存。

因此需要考慮：

- Calories
- Protein
- Carbohydrates
- Fat
- Vitamins
- Minerals
- Dietary diversity

例如 Lettuce 可能：

- 生長快速
- 水需求較低
- 提供部分維生素

但 calorie density 很低。

Potato 則可能：

- 熱量較高
- 適合作為主要 calorie source
- 但營養組成與資源需求不同

因此問題變成：

如何在有限種植空間、水與能源下，建立可以長期滿足 Crew 營養需求的 Crop Portfolio？

3.3 Circular Life Support

系統中的資源不是單向消耗，而是形成循環。

例如：

```
Human
↓
CO2
↓
Plants
↓
O2 + Food
↓
Human
```

水資源：

```
Human / Plant
↓
Wastewater
↓
Water Treatment
↓
Irrigation
↓
Plant
```

有機物：

```
Food / Plant Waste
↓
Biological Processing
↓
Nutrient Recovery
↓
Crop
```

因此系統管理的已經不只是 Greenhouse，而是一個：

Closed-Loop Artificial Ecosystem

4. 為什麼不是一般 AIoT Smart Greenhouse？

兩者最大的差異不在 Sensor，而在「決策問題」。

	AIoT Smart Greenhouse	Proposed System
核心目標	維持植物最佳環境	維持整個 Mission
Input	Sensor data	Sensor + Resources + Forecast + Crew + Mission
Decision	Local control	Global planning
時間尺度	即時反應	Hours → Days → Months
問題	Detect abnormality	Decide strategy
資源	通常可補充	極度有限
Objective	Crop health / Yield	Human survival + Food + Resource
AI Role	Detection / Prediction	Planning / Reasoning / Adaptation
系統行為	Reactive	Proactive

傳統 AIoT 的核心較接近：

```

Temperature > threshold
    ↓
Cooling ON

```

本專案則可能面對：

```

Temperature      31°C
Humidity         85%
Battery          23%
Solar Forecast   -50%
Water Reserve    31%
Crop Health      87%

Future Mission
Remaining        287 days

```

此時：

- Cooling 需要能源
- Dehumidification 需要能源
- LED 需要能源
- Water recycling 需要能源
- Life support 也需要能源

因此不能單純：

「熱 → Cooling ON」

而需要回答：

有限能源到底應該分配到哪裡？

5. Agentic AI 的角色

本專案不希望將所有工程規則寫成：

```
IF battery < 20%  
THEN reduce LED by 30%
```

因為這仍然只是 predefined automation。

Agent 的核心責任應該是：

在 Goal、Constraints、Available Tools 已知的情況下，自主尋找目前情境下適合的策略。

6. 哪些東西可以固定？

Agentic AI 並不代表所有事情都由 LLM 決定。

系統仍然需要固定：

6.1 Environment / Physical Rules

例如：

- 植物生長模型
- 能源消耗模型
- 水循環模型
- O₂ / CO₂ balance
- Battery model
- Crop nutrition model

這些描述的是：

世界如何運作。

而不是：

Agent 應該做什麼。

6.2 Action Space

Agent 只能使用實際存在的工具。

例如：

```
set_led(crop, intensity)

set_temperature(value)

set_irrigation(crop, amount)

allocate_energy(system, percentage)

plant(crop, area)

harvest(crop)

switch_backup_system(device)

suspend_equipment(device)
```

6.3 Hard Constraints

部分生命安全條件不可被 Agent 任意違反。

例如：

- Crew O₂ 不可低於安全值
- Battery 必須保留 emergency reserve
- 飲用水不可低於最低需求
- Crew calorie supply 不可長期低於生存需求

這些屬於：

Safety Constraints

而不是 Agent Policy。

7. Agent 不應該被寫死的部分

不應預先設定：

```
Solar storm
→ LED -30%
```

而應讓 Agent 自己決定：

「在這一次 Solar Storm 的資源狀況下，LED 應該降低多少？」

因此：

Objective 可以固定。

但：

Policy 不應固定。

8. Agentic Decision Loop

系統核心流程可以設計為：

```
graph TD;
  A[Observe] --> B[Understand Situation];
  B --> C[Identify Problems];
  C --> D[Form Hypotheses];
  D --> E[Generate Strategy];
  E --> F[Use Tools / Simulation];
  F --> G[Evaluate Future Consequences];
  G --> H[Revise Strategy];
  H --> I[Execute];
  I --> J[Observe Result];
  J --> K[Reflect];
  K --> A;
```

與傳統 Automation：

```
graph TD;
  A[Condition] --> B[Rule];
  B --> C[Action];
```

形成明顯區隔。

9. Digital Twin / Simulator

Digital Twin 不負責「告訴 Agent 正確答案」。

它負責回答：

如果你真的這樣做，未來可能會發生什麼？

例如目前發生 Solar Storm：

Battery	34%
Solar Generation	-50%
Duration	72h

Agent自行提出：

Strategy A

LED -20%

Simulator：

Battery after 72h = -4%

MISSION UNSAFE

Agent發現策略不可行。

接著自行提出：

Strategy B

Tomato LED -50%
Lettuce LED -20%
Temperature -1.5°C
Suspend non-critical equipment

Simulator：

Battery after 72h = 16%
Crop Health = 86%
Food Security = 93%

MISSION SAFE

Agent可以繼續尋找更好的策略。

10. Multi-Agent Architecture

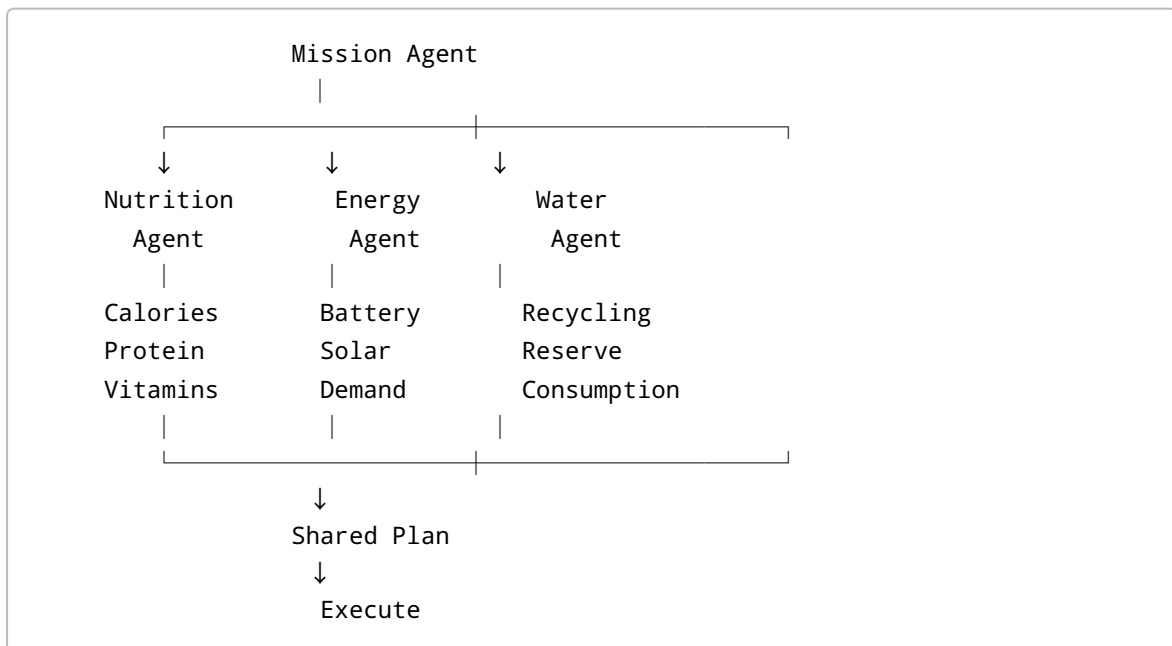
Multi-Agent 的目的不是：

一個 Sensor 配一個 Agent。

而是：

不同領域擁有不同 Knowledge、Objective 與 Constraints，需要共同形成全局策略。

例如：



11. Agent Conflict Example

假設發生 Solar Storm。

Nutrition / Crop Agent

希望：

保留更多 LED，以避免 Food Production 大幅下降。

Energy Agent

認為：

目前策略將使 Battery 在 38 小時後低於 Emergency Reserve。

Water Agent

提出：

可以提高 ventilation 代替部分 cooling。

但發現：

Plant transpiration 增加，將使未來三天 Water Demand +17%。

Mission Agent

因此需要綜合：

- Food Security
- Energy Reserve
- Water Reserve
- Crew Safety
- Crop Health
- Future Mission Duration

形成最終策略。

這不是單純：

Sensor → Response

而是：

Multiple Objectives → Conflict → Reasoning → Planning → Trade-off

12. Agent Memory

Agent 可以保留過去 Mission 中的經驗。

例如 Day 41 發生能源危機：

Incident:
Solar generation -45%

Strategy:
Reduce tomato lighting before lettuce.

Result:
Energy consumption -31%
Calories production -8%
Mission remained stable.

系統形成：

Lesson:

During severe energy shortages,
reducing tomato lighting before lettuce
produced a better calorie-per-kWh trade-off.

Day 187 再遇到類似事件時：

```
Current Incident
  ↓
Retrieve Similar Experience
  ↓
Previous Strategy
  ↓
Adapt to Current State
  ↓
Simulation
  ↓
New Strategy
```

Memory 是參考，而不是固定規則。

Agent 仍然需要根據目前狀況重新驗證。

13. Hackathon Demo Scenario

Demo 可以設定：

Mission

Mars Habitat

Crew	4
Mission Duration	500 days
Growing Area	40 m ²
Water	Limited
Energy	Solar + Battery
Earth Resupply	None

Available Crops :

- Potato
- Soybean
- Wheat
- Lettuce
- Tomato

Phase 1 — Mission Planning

使用者輸入：

Crew: 4
Mission: 500 days
Growing Area: 40 m²

Agent自主產生：

- Crop allocation
- Planting schedule
- Harvest schedule
- Water allocation
- Energy allocation
- Emergency reserve strategy

Phase 2 — Normal Operation

Dashboard 顯示：

MISSION DAY 137 / 500

Crew Safety	SAFE
Food Security	94%
Water Reserve	72%

Energy Reserve	68%
Crop Health	91%

Phase 3 — Emergency

使用者按下：

INJECT EMERGENCY

事件：

SOLAR STORM

Solar Generation
-58%

Expected Duration
72 hours

Phase 4 — Autonomous Replanning

Agent自行：

1. 分析目前狀態
2. 找出主要風險
3. 呼叫不同領域 Agent
4. 產生候選策略
5. 使用 Simulator 測試
6. 比較未來結果
7. 修改策略
8. 選擇方案
9. 執行
10. 持續監控

Dashboard 可以視覺化整個 Reasoning / Planning 過程。

14. Evaluation

Agent不應該以「植物長得最好」作為唯一評分標準。

核心 Objective 應包含：

- Crew Survival

- Food Security
- Nutritional Coverage
- Water Efficiency
- Energy Efficiency
- Crop Health
- System Resilience
- Mission Completion

並設定 Safety Constraints 。

例如：

Mission Completion	Highest Priority
Crew Safety	Hard Constraint
Nutrition	High
Water Reserve	High
Energy Reserve	High
Crop Health	Medium
Comfort	Low

因此 Agent 必須學會：

在不同目標之間進行 trade-off 。

15. Baseline Comparison

為證明 Agentic AI 確實產生價值，可以建立 Rule-Based AIoT Controller 作為 Baseline 。

Rule-Based

```

Detect
↓
Threshold / Prediction
↓
Predefined Response
↓
Execute

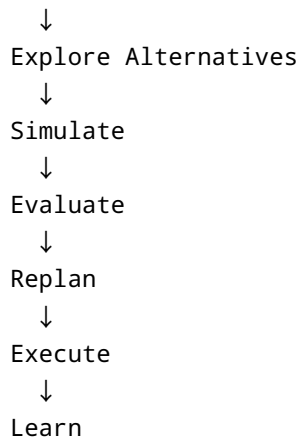
```

Agentic System

```

Observe
↓
Reason
↓
Plan

```



透過多個 randomized mission scenarios 比較：

- Mission survival rate
- Resource consumption
- Food security
- Nutritional coverage
- Emergency recovery time
- Number of safety violations
- Mission completion rate

從而驗證：

Agent 是否真的比 predefined automation 更能應對未知與複合型事件。

16. 專案核心價值

本專案最終不是：

「AI 幫忙管理太空溫室。」

而是：

「AI 自主規劃並維持一個支撐人類長期生存的封閉資源生態系統。」

太空溫室是整個系統的核心，但問題已經延伸至：

Space Agriculture

+ **Human Nutrition**

+ **Circular Life Support**

+ **Resource Optimization**

再透過：

Agentic AI

+ **Multi-Agent System**

+ **Digital Twin / Simulation**

解決跨領域之間互相衝突的決策問題。

17. 一句話 Project Pitch

What if Earth could no longer help?

建立一個能夠自主觀察、規劃、模擬、決策與調整的 AI 系統，在有限的水、能源、食物與種植資源下，自主維持太空基地中的人類與植物生態系統。

核心不是讓 AI 「控制溫室」。

而是讓 AI 回答：

How do we keep humans alive when every resource is limited?